

Mineração de Interações GitHub e Implementação de Grafos em Python

Amanda Bicalho Silva ¹, Karen Joilly Araújo Gregório de Almeida ¹,
Pedro Rodrigues Duarte ¹, Tiago Boaventura Amaral ¹

¹Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

{amanda.bicalho, pedro.duarte.1442616, tiago.amaral.1515281}@sga.pucminas.br

{karenjoilly}@gmail.com

Resumo. *Este trabalho apresenta o desenvolvimento de um sistema de mineração de dados aplicado ao GitHub, utilizando a linguagem Python e conceitos da Teoria dos Grafos. O projeto realiza a coleta de interações entre usuários por meio da análise de issues e comentários do repositório Typer, transformando essas relações em grafos direcionados que representam a comunicação entre os contribuidores. Para a representação dos grafos, foram implementadas manualmente as estruturas de lista de adjacência e matriz de adjacência, sem a utilização de bibliotecas especializadas para manipulação de grafos. Além disso, foram desenvolvidas funcionalidades para verificação de conectividade por meio do algoritmo Depth First Search (DFS), cálculo do grau dos vértices, contagem de arestas, tratamento de exceções, exportação dos dados para visualização no software Gephi e benchmark comparativo entre as estruturas implementadas. Os resultados obtidos demonstraram a viabilidade da utilização de grafos para representar interações reais em plataformas colaborativas de desenvolvimento de software, possibilitando a análise de relações entre usuários e a comparação de diferentes estratégias de representação de grafos. Dessa forma, o trabalho integra conceitos de mineração de dados, programação orientada a objetos e análise de redes, proporcionando uma aplicação prática da Teoria dos Grafos em um cenário real.*

Palavras-chave: *Teoria dos Grafos, Mineração de Dados, GitHub, Python, Gephi, Redes Sociais.*

1. Introdução

A Teoria dos Grafos representa uma importante área da Ciência da Computação e da Matemática Discreta, sendo amplamente aplicada em problemas relacionados à representação de conexões, modelagem de redes e análise estrutural de relacionamentos entre entidades. Um grafo é composto por vértices e arestas, permitindo representar diferentes tipos de interações presentes em sistemas computacionais, redes sociais, sistemas distribuídos, mecanismos de recomendação, rotas de navegação, análise de dependências e mineração de dados.

Com a expansão das plataformas colaborativas de desenvolvimento de software, especialmente o GitHub, tornou-se possível estudar relações sociais e técnicas entre desenvolvedores através de interações registradas em issues, comentários, pull requests e

contribuições realizadas em projetos open source. Essas plataformas armazenam grandes volumes de informações que podem ser mineradas e modeladas utilizando grafos, permitindo identificar padrões de comunicação, colaboração e participação entre usuários.

A mineração de repositórios de software tornou-se uma relevante área de pesquisa dentro da Engenharia de Software, possibilitando compreender comportamentos de equipes de desenvolvimento, analisar estruturas de colaboração e identificar características de projetos de código aberto. Nesse contexto, o GitHub disponibiliza uma API oficial que permite acessar dados públicos de repositórios, contribuidores, comentários e demais interações realizadas na plataforma.

Nesse contexto, este trabalho propõe o desenvolvimento de um sistema de mineração de dados utilizando Python e a API do GitHub para coletar interações reais entre usuários do repositório Typer [FASTAPI 2026], um projeto open source amplamente utilizado para construção de interfaces de linha de comando em Python. As interações mineradas foram transformadas em grafos simples, nos quais os usuários representam vértices e os comentários representam arestas direcionadas entre os participantes das discussões.

Além da etapa de mineração dos dados, o projeto implementa manualmente duas representações clássicas de grafos: lista de adjacência e matriz de adjacência. O desenvolvimento foi realizado sem a utilização de bibliotecas prontas para estruturação de grafos, permitindo compreender profundamente os conceitos fundamentais envolvidos na manipulação dessas estruturas.

Também foram desenvolvidas funcionalidades relacionadas à Teoria dos Grafos, incluindo verificação de conectividade utilizando busca em profundidade (DFS), cálculo de grau dos vértices, exportação de grafos para visualização no software Gephi [CONSORTIUM 2026] e benchmark comparativo entre as estruturas implementadas. O projeto ainda contempla tratamento de exceções, controle de loops e prevenção de arestas paralelas, respeitando as características de grafos simples.

Dessa forma, o trabalho integra conceitos teóricos e práticos de mineração de dados, programação orientada a objetos, estruturas de dados e análise de redes, permitindo aplicar a Teoria dos Grafos em um contexto real de interação social entre desenvolvedores de software.

2. Objetivo

O presente trabalho tem como objetivo desenvolver um sistema de mineração de dados aplicado ao GitHub, capaz de coletar e representar interações entre usuários por meio de estruturas de grafos implementadas na linguagem Python. A partir da análise de issues e comentários do repositório Typer, busca-se modelar as relações existentes entre os contribuidores, transformando essas interações em grafos simples compostos por vértices e arestas direcionadas.

Além da etapa de mineração dos dados, o trabalho tem como objetivo implementar e comparar diferentes formas de representação de grafos, especificamente lista de adjacência e matriz de adjacência, analisando suas características e desempenho. Também pretende aplicar conceitos fundamentais da Teoria dos Grafos, incluindo cálculo do grau

dos vértices, verificação de conectividade por meio de busca em profundidade (DFS), tratamento de exceções e exportação dos dados para ferramentas de visualização.

Por fim, busca-se demonstrar a aplicabilidade da Teoria dos Grafos em um contexto real de colaboração em software, permitindo analisar relações entre usuários, comparar estruturas de representação e visualizar as interações obtidas por meio do software Gephi.

3. Justificativa

A crescente utilização de plataformas colaborativas de desenvolvimento de software tem gerado grandes volumes de dados relacionados à comunicação e à colaboração entre desenvolvedores. Entre essas plataformas, o GitHub destaca-se como uma das mais utilizadas pela comunidade de software livre, reunindo milhões de usuários e armazenando informações sobre issues, comentários, pull requests e diversas outras formas de interação. A análise desses dados possibilita compreender padrões de colaboração, identificar usuários influentes e investigar a estrutura das relações existentes dentro de projetos de desenvolvimento.

Nesse contexto, a Teoria dos Grafos apresenta-se como uma ferramenta adequada para modelar e analisar redes de interação, permitindo representar usuários como vértices e suas relações como arestas. A utilização de grafos possibilita estudar propriedades estruturais da rede, identificar comunidades, analisar centralidade de participantes e compreender a dinâmica de comunicação existente em ambientes colaborativos.

Além da relevância prática da análise de redes sociais aplicadas à Engenharia de Software, o projeto possui importância acadêmica por permitir a aplicação concreta dos conceitos estudados na disciplina de Teoria dos Grafos. Diferentemente de abordagens que utilizam bibliotecas prontas para manipulação de grafos, este trabalho propõe a implementação manual das estruturas de lista de adjacência e matriz de adjacência, possibilitando uma compreensão mais aprofundada dos mecanismos internos de armazenamento, manipulação e análise dessas estruturas.

A escolha do repositório Typer como objeto de estudo também se justifica pela sua relevância no ecossistema Python e pela significativa quantidade de interações entre contribuidores. A mineração de dados realizada sobre um projeto real permite aproximar os conceitos teóricos da realidade encontrada em ambientes de desenvolvimento colaborativo, tornando a análise mais representativa e enriquecedora.

Adicionalmente, o desenvolvimento do projeto integra conhecimentos de diferentes áreas da Computação, incluindo Programação Orientada a Objetos, Estruturas de Dados, Mineração de Dados, Engenharia de Software e Análise de Redes. Essa integração contribui para a formação acadêmica dos autores ao proporcionar experiência prática na utilização de APIs, tratamento de dados, modelagem computacional e visualização de informações.

Dessa forma, o trabalho justifica-se tanto pela sua contribuição para o aprendizado dos conceitos fundamentais da Teoria dos Grafos quanto pela aplicação desses conceitos em um cenário real de colaboração em software, demonstrando como técnicas de mineração de dados e análise de redes podem ser utilizadas para extrair conhecimento a partir de interações registradas em plataformas modernas de desenvolvimento.

4. Metodologia

O desenvolvimento do projeto foi realizado utilizando a linguagem Python, escolhida devido à sua ampla utilização em aplicações de mineração de dados, facilidade de integração com serviços externos e disponibilidade de bibliotecas para processamento de informações. Para a coleta dos dados foi utilizada a biblioteca PyGithub, que fornece acesso à API oficial do GitHub por meio de autenticação baseada em token, permitindo consultar repositórios, usuários, issues e comentários de forma estruturada.

A etapa inicial consistiu na mineração de dados do repositório Typer [FASTAPI 2026], selecionado devido à sua relevância para o ecossistema Python e à elevada quantidade de colaboradores e interações registradas. O processo de mineração percorreu as issues fechadas do repositório e seus respectivos comentários, identificando relações entre os autores das issues e os usuários responsáveis pelas interações. Cada interação identificada foi transformada em uma aresta direcionada do grafo, representando a comunicação entre dois colaboradores.

Os dados coletados foram armazenados em arquivos no formato `.gphi`, possibilitando a persistência das informações obtidas e sua reutilização nas etapas posteriores de construção, processamento e análise dos grafos. Cada registro contém o usuário de origem, o usuário de destino e o peso associado à interação observada.

A modelagem adotada representa cada colaborador como um vértice e cada interação como uma aresta direcionada. Além da representação simples, foi utilizada uma abordagem ponderada, na qual o peso de cada aresta representa a intensidade da relação entre dois usuários. Sempre que uma nova interação entre os mesmos colaboradores era identificada durante a mineração, o peso da conexão correspondente era incrementado. Dessa forma, relações mais frequentes passaram a possuir maior relevância na estrutura da rede.

Após a coleta dos dados, foram implementadas manualmente duas estruturas clássicas de representação de grafos: lista de adjacência e matriz de adjacência. Foram implementadas as representações por lista de adjacência e matriz de adjacência, permitindo comparar diferentes estratégias de armazenamento e processamento dos grafos.

O sistema foi desenvolvido seguindo princípios de Programação Orientada a Objetos, empregando conceitos de abstração, herança, encapsulamento e modularização. Para padronizar as operações comuns entre as diferentes representações, foi criada uma classe abstrata denominada `AbstractGraph`, responsável por definir a interface utilizada pelas implementações concretas de lista e matriz de adjacência.

Além das operações básicas de inserção e remoção de arestas, foram implementadas funcionalidades para verificação de conectividade, consulta de predecessores e sucessores, cálculo de graus, manipulação de pesos e exportação dos dados para ferramentas externas. A análise de conectividade foi realizada utilizando o algoritmo Depth First Search (DFS), responsável por percorrer recursivamente os vértices alcançáveis a partir de um vértice inicial.

Considerando os requisitos do trabalho, também foram implementadas validações para garantir que os grafos permanecessem simples, impedindo a criação de laços e arestas paralelas. Essas verificações foram realizadas durante as operações de inserção de arestas,

assegurando a consistência da estrutura de dados.

Com o objetivo de avaliar o desempenho das implementações desenvolvidas, foram realizados testes de benchmark comparando a lista de adjacência e a matriz de adjacência. Os experimentos mediram principalmente o tempo de inserção de arestas e o comportamento das estruturas diante de diferentes volumes de dados, permitindo analisar suas vantagens e limitações em cenários distintos.

A análise estrutural da rede permitiu identificar diversas propriedades relevantes. O grafo integrado apresentou 1129 vértices e 1952 arestas. O grau médio observado foi de 1,557, indicando que a maioria dos colaboradores possui poucas conexões diretas com outros participantes da rede. A densidade observada foi de 0,0015, caracterizando uma rede bastante esparsa, comportamento comum em redes sociais e projetos de software livre.

Por fim, os grafos gerados foram exportados para arquivos e importados para o software Gephi [CONSORTIUM 2026]. A ferramenta foi utilizada para a visualização da rede e para o cálculo de métricas estruturais, incluindo grau médio, grau ponderado, densidade, diâmetro, modularidade, componentes conectados, PageRank, HITS, centralidade de autovetor e coeficiente de clustering. Essas análises permitiram compreender a estrutura da rede de colaboração formada pelos usuários do repositório e identificar padrões de interação entre seus participantes.

5. Modelagem da Solução

A Figura 1 apresenta o diagrama de classes desenvolvido para o projeto. O diagrama representa a estrutura geral do sistema, incluindo a classe abstrata responsável pela definição das operações básicas dos grafos, as implementações de lista de adjacência e matriz de adjacência, além das classes auxiliares utilizadas para benchmark, carregamento de dados e tratamento de exceções.

A modelagem orientada a objetos permitiu maior organização e modularização do projeto, facilitando a reutilização de código e a padronização das operações realizadas sobre os grafos.

6. Resultados

Os resultados obtidos demonstraram que o sistema foi capaz de minerar corretamente interações reais entre usuários do GitHub e representá-las através de estruturas de grafos implementadas manualmente em Python. A mineração realizada no repositório Typer permitiu identificar relações entre contribuidores a partir de comentários em issues fechadas, gerando um conjunto de dados válido para construção dos grafos.

A implementação das estruturas de lista de adjacência e matriz de adjacência apresentou funcionamento adequado em todas as operações desenvolvidas. Os testes realizados comprovaram o correto funcionamento das funcionalidades de inserção e remoção de arestas, verificação de conectividade, cálculo de grau dos vértices e contagem de arestas. Os testes de exceções também demonstraram que o sistema respeita corretamente as restrições de grafos simples, impedindo a criação de loops e evitando comportamentos inválidos durante a manipulação das estruturas.

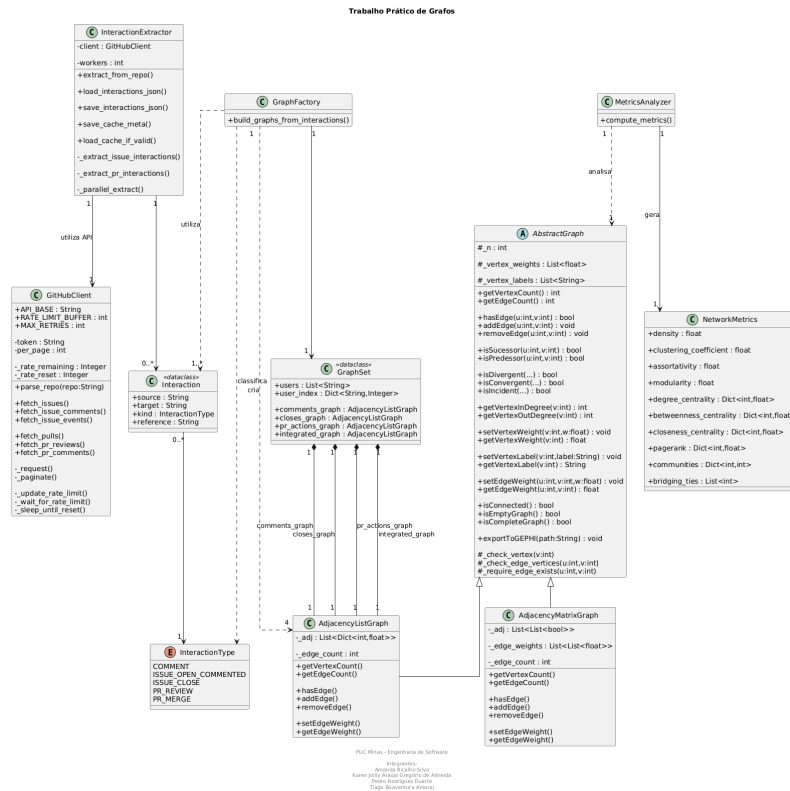


Figura 1. Diagrama de classes do sistema

Durante os benchmarks realizados, ambas as estruturas apresentaram desempenho satisfatório para os volumes de dados utilizados. A matriz de adjacência apresentou tempo de acesso ligeiramente mais rápido em determinadas operações, enquanto a lista de adjacência demonstrou menor consumo de memória e maior flexibilidade para grafos esparsos.

A exportação dos dados para arquivos permitiu integração com o software Gephi, possibilitando a visualização gráfica das relações entre usuários. A visualização gerada evidenciou a existência de conexões entre contribuidores do repositório analisado, permitindo observar padrões básicos de interação e colaboração.

Além dos resultados técnicos, o projeto proporcionou uma aplicação prática dos conceitos estudados em Teoria dos Grafos, mineração de dados e estruturas de dados, permitindo compreender diferenças entre representações de grafos e desafios envolvidos na manipulação dessas estruturas sem o uso de bibliotecas prontas. O desenvolvimento do trabalho também contribuiu para o aprendizado relacionado à organização de projetos em Python, modularização de código, tratamento de exceções, utilização de APIs e integração com ferramentas externas de visualização de dados.

6.1. Comparação entre Lista de Adjacência e Matriz de Adjacência

Durante os testes realizados, foram analisadas as duas estruturas de representação de grafos implementadas no projeto: lista de adjacência e matriz de adjacência. Ambas apresentaram funcionamento correto para as operações de inserção de arestas, cálculo de grau, verificação de conectividade e manipulação da rede construída a partir dos dados

minerados do GitHub.

Segundo Cormen et al. (2022), a lista de adjacência é mais eficiente para grafos esparsos, enquanto a matriz de adjacência oferece consultas de adjacência em tempo constante [CORMEN et al. 2022]. Logo, isso foi possível comprovar quando os resultados obtidos evidenciaram diferenças importantes entre as duas abordagens. A lista de adjacência apresentou menor consumo de memória, uma vez que armazena apenas os vértices efetivamente conectados. Essa característica torna a estrutura especialmente adequada para grafos esparsos, como o grafo gerado neste trabalho, cuja densidade foi igual a 0,002.

Por outro lado, a matriz de adjacência apresentou maior rapidez na verificação da existência de arestas entre dois vértices, já que a consulta pode ser realizada diretamente por meio dos índices da matriz. Entretanto, essa vantagem ocorre ao custo de um maior consumo de memória, pois é necessário armazenar todas as possíveis conexões entre os vértices, independentemente de existirem ou não.

Dessa forma, os experimentos demonstraram que não existe uma estrutura universalmente superior. A escolha entre lista de adjacência e matriz de adjacência depende das características do problema analisado. Para redes esparsas e com grande quantidade de vértices, a lista de adjacência tende a apresentar melhor eficiência em memória. Já em aplicações que exigem consultas frequentes de adjacência, a matriz pode oferecer melhor desempenho computacional.

6.2. Visualização da Rede

6.2.1. Grafo de Comentários

O grafo de comentários representa as interações realizadas por meio de comentários em *Issues* e *Pull Requests*. Observa-se uma rede relativamente conectada, composta por um núcleo central de usuários altamente ativos e diversos colaboradores periféricos. A presença de 51 componentes conectados indica que a comunicação ocorre principalmente em torno de um conjunto reduzido de participantes centrais.

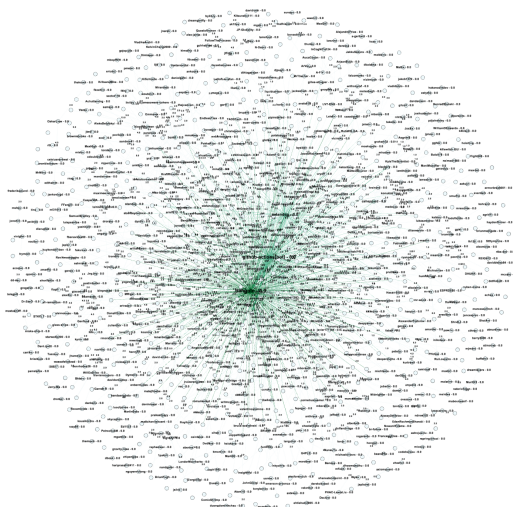


Figura 2. Rede de interações por meio de comentários do repositório Typer.

6.2.2. Grafo de Fechamentos

O grafo de fechamentos representa as relações entre usuários responsáveis pelo encerramento de *Issues* e *Pull Requests*. Observa-se uma estrutura altamente centralizada, na qual poucos usuários concentram a maior parte das atividades de fechamento. O elevado número de componentes conectados evidencia que essa atividade é desempenhada por um grupo restrito de mantenedores do projeto.

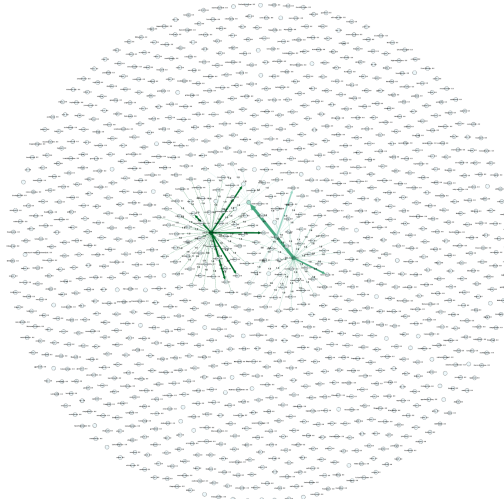


Figura 3. Rede de interações por fechamentos de Issues e Pull Requests.

6.2.3. Grafo de Ações em Pull Requests

O grafo de ações em *Pull Requests* representa as interações relacionadas à abertura, aprovação e gerenciamento de contribuições ao projeto. A rede apresenta poucos usuários centrais e grande quantidade de participantes periféricos, demonstrando que as atividades de coordenação estão concentradas em um pequeno grupo de colaboradores.

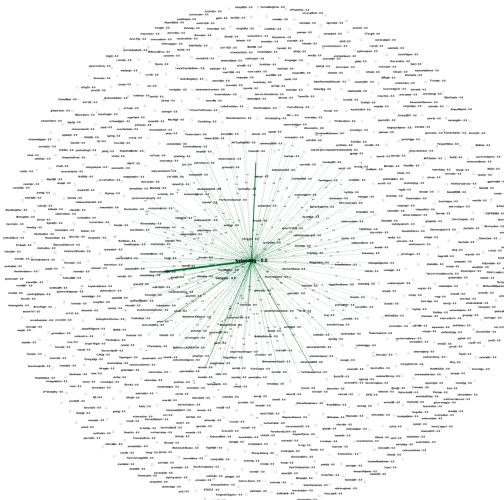


Figura 4. Rede de interações relacionadas às ações em Pull Requests.

6.2.4. Grafo Integrado

O grafo integrado combina as informações dos grafos de comentários, fechamentos e ações em *Pull Requests*, fornecendo uma visão abrangente das interações existentes no repositório. Observa-se uma rede mais conectada, com menor quantidade de componentes isolados e maior concentração de conexões em torno dos principais colaboradores do projeto.

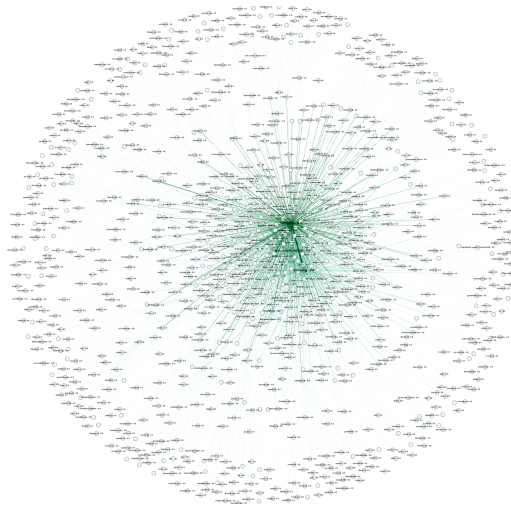


Figura 5. Grafo integrado contendo todas as interações mineradas do repositório Typer.

6.3. Métricas Estruturais

A análise estrutural da rede permitiu identificar diversas propriedades relevantes. O grafo integrado apresentou 1129 vértices e 1952 arestas. A densidade observada foi de 0,0015, indicando uma rede bastante esparsa, característica comum em redes sociais e projetos de software livre. O coeficiente médio de clustering foi igual a 0,0215, evidenciando baixa formação de agrupamentos locais, indicando uma rede bastante esparsa, característica comum em redes sociais e projetos de software livre.

O diâmetro da rede foi igual a 7 e o comprimento médio dos caminhos foi de aproximadamente 2,963. Esses resultados indicam que a comunicação entre colaboradores ocorre por meio de poucos intermediários, refletindo a propriedade de mundo pequeno frequentemente observada em redes sociais.

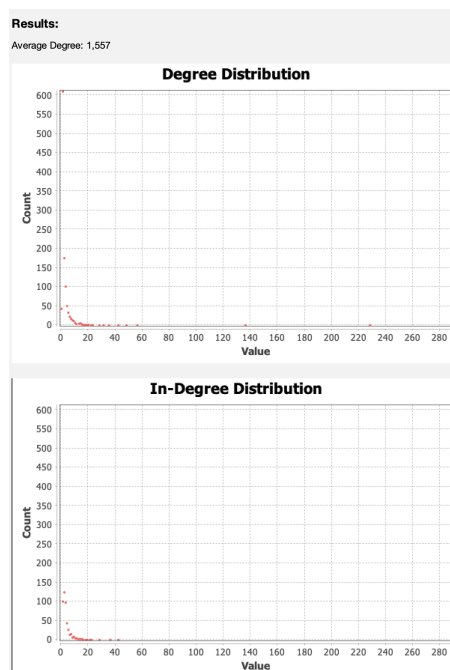
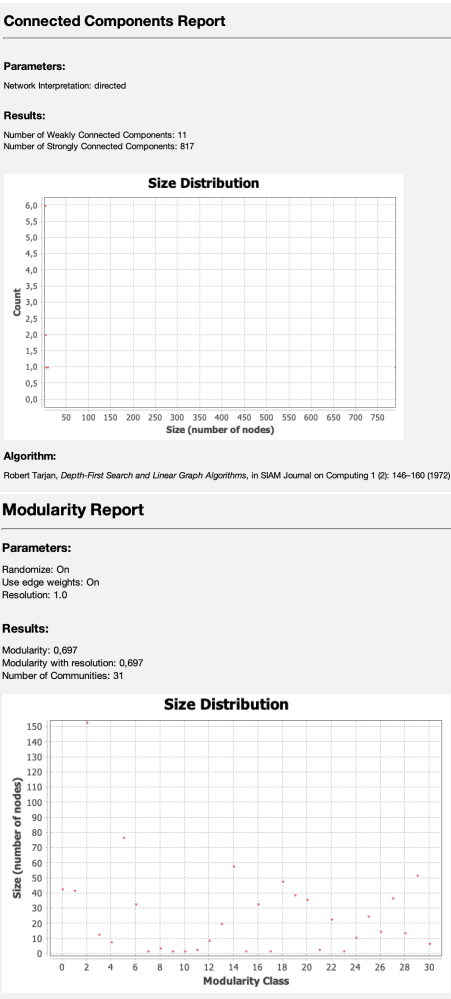


Figura 6. Distribuição dos graus da rede.

A Figura 6 apresenta as distribuições dos graus da rede. Observa-se que a maioria dos usuários possui poucas conexões, enquanto um número reduzido de vértices concentra grande parte das interações.

6.4. Comunidades e Componentes

A análise de componentes conectados identificou 7 componentes na rede integrada. O algoritmo de detecção de comunidades obteve modularidade igual a 0,508. Esse resultado indica a existência de grupos de colaboradores que interagem mais frequentemente entre si do que com o restante da rede, embora a separação entre comunidades seja moderada.



6.5. Centralidade dos Usuários

A análise das métricas de centralidade permitiu identificar os usuários mais relevantes da rede de interações do repositório Typer. Foram utilizadas as métricas PageRank, HITS (Authorities e Hubs), Centralidade de Autovetor e Closeness Centrality, cada uma destacando diferentes aspectos da importância estrutural dos vértices.

A métrica Degree Centrality apontou o usuário *tiangolo* como o participante com maior número de conexões diretas na rede, seguido por *github-actions[bot]*, *svlandeg*, *YuriiMotov* e *codecov[bot]*. Esse resultado evidencia a forte participação desses usuários nas atividades do projeto e sua relevância na estrutura de colaboração.

O algoritmo PageRank identificou o usuário *csheppard* como o mais influente da rede. Em seguida destacaram-se os usuários *bcm0*, *tiangolo*, *svlandeg* e *fschlz*. Esses participantes ocupam posições estratégicas na estrutura da rede, recebendo conexões provenientes de usuários relevantes.

A métrica HITS (Hyperlink-Induced Topic Search) foi utilizada para identificar usuários que atuam como hubs e authorities dentro da rede. Os resultados indicam uma distribuição fortemente concentrada, na qual a maioria dos colaboradores apresenta valores próximos de zero, enquanto um pequeno grupo de participantes concentra os maiores escores.

Os usuários classificados como hubs representam participantes que interagem com diversos colaboradores relevantes da rede. Já os usuários classificados como authorities correspondem àqueles frequentemente referenciados ou envolvidos em interações provenientes de participantes importantes. Esse comportamento evidencia uma estrutura típica de projetos de software livre, caracterizada pela concentração das interações em torno de um grupo reduzido de colaboradores centrais.

A centralidade de autovetor apresentou comportamento semelhante ao observado no PageRank, destacando usuários localizados em posições estratégicas da rede e conectados a outros participantes relevantes. Essa métrica reforça a existência de um núcleo central de colaboradores responsáveis por grande parte das interações do projeto.

A centralidade de intermediação (Betweenness Centrality) destacou os usuários *tiangolo*, *svlandeg*, *github-actions[bot]* e *YuriiMotov*. Esses participantes atuam como pontes entre diferentes regiões da rede, facilitando a circulação de informações entre grupos distintos de colaboradores.

A análise das comunidades permitiu observar a existência de usuários que conectam diferentes grupos identificados pelo algoritmo de modularidade. Esses vértices exercem papel de bridging ties, atuando como elo entre comunidades distintas e contribuindo para a integração da rede.

A métrica de Closeness Centrality foi analisada para avaliar a proximidade dos usuários em relação aos demais participantes da rede. Os resultados indicam que os colaboradores centrais conseguem alcançar outros vértices por meio de caminhos relativamente curtos, ocupando posições privilegiadas na estrutura de colaboração.

De forma geral, os resultados das métricas de centralidade demonstram uma concentração das interações em torno de um grupo reduzido de usuários, característica comum em projetos de software livre, nos quais poucos colaboradores exercem papel

central na coordenação das atividades e na disseminação de informações dentro da comunidade.

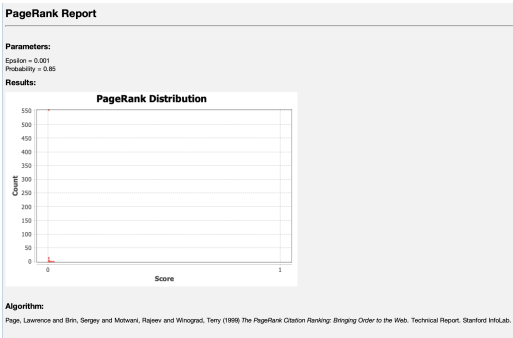


Figura 8. Distribuição dos valores de PageRank.

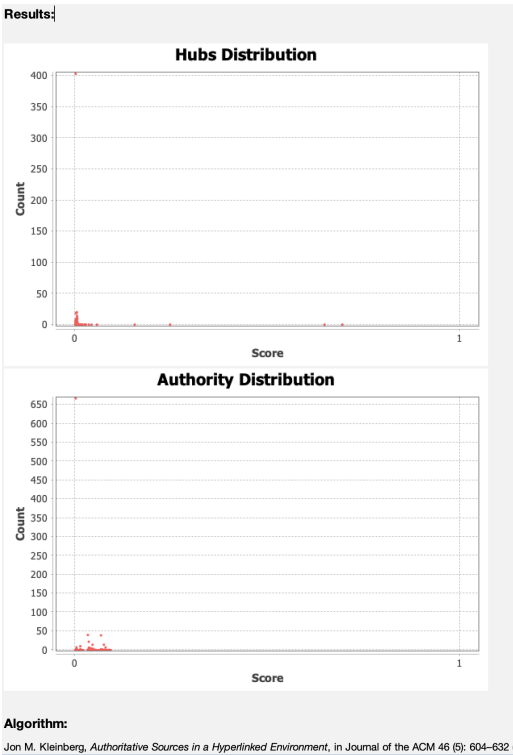


Figura 9. Distribuição dos valores de HITS.

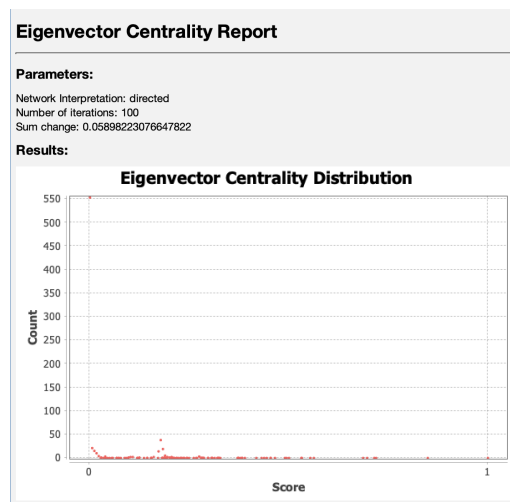


Figura 10. Distribuição da centralidade de autovetor.

6.6. Assortatividade

A assortatividade da rede foi igual a $-0,1695$, indicando comportamento desassortativo. Isso significa que usuários altamente conectados tendem a interagir com colaboradores que possuem poucas conexões. Esse padrão é comum em projetos de software livre, nos quais mantenedores centrais interagem frequentemente com novos contribuidores e participantes ocasionais.

6.7. Bridging Ties

A análise das comunidades identificou os usuários *tiangolo*, *svlandeg*, *YuriiMotov*, *csheppard* e *bcm0* como principais bridging ties da rede. Esses participantes conectam diferentes comunidades de colaboradores, contribuindo para a integração global do projeto.

6.8. Coeficiente de Clustering

O coeficiente médio de clustering obtido foi igual a $0,0215$. Esse valor indica baixa formação de triângulos e agrupamentos locais, sugerindo que a maioria das interações ocorre entre colaboradores centrais e usuários periféricos, sem a formação de grupos altamente coesos.

6.9. Materiais Complementares

- Repositório: <https://github.com/TiagoBoavAmaral/trabalho-grafos-typer.git>
- Vídeo: disponível na pasta `Documentacao/ApresentacaoVideo.mp4`
- Apresentação: disponível na pasta `Documentacao/Slides.pdf`

7. Conclusões e trabalhos futuros

O presente trabalho atingiu o objetivo proposto de desenvolver um sistema de mineração de dados aplicado ao GitHub, capaz de coletar interações reais entre usuários e representá-las por meio de estruturas de grafos implementadas na linguagem Python. A partir da análise do repositório Typer, foi possível construir uma rede de colaboração

representativa, permitindo aplicar conceitos da Teoria dos Grafos em um cenário real de desenvolvimento de software.

As análises realizadas evidenciaram características importantes da rede estudada. As métricas estruturais demonstraram que a rede possui baixa densidade e forte concentração das interações em torno de um grupo reduzido de usuários. As métricas de centralidade permitiram identificar colaboradores mais influentes e relevantes para a comunicação do projeto, enquanto a análise de modularidade revelou a existência de comunidades bem definidas dentro da rede de colaboração.

A comparação entre as representações por lista de adjacência e matriz de adjacência possibilitou observar, na prática, as vantagens e limitações de cada abordagem. Os resultados obtidos reforçaram a importância da escolha adequada da estrutura de dados de acordo com as características do problema analisado, evidenciando o melhor aproveitamento da lista de adjacência em grafos esparsos e a eficiência da matriz de adjacência em consultas diretas de conectividade.

Além dos resultados técnicos, o projeto contribuiu significativamente para a consolidação dos conhecimentos relacionados à Teoria dos Grafos, Estruturas de Dados, Programação Orientada a Objetos, Mineração de Dados e Análise de Redes. A utilização de dados reais obtidos por meio da API do GitHub proporcionou uma experiência prática e enriquecedora, aproximando os conceitos teóricos estudados em sala de aula de aplicações encontradas em ambientes reais de desenvolvimento colaborativo.

Como trabalhos futuros, pretende-se ampliar a mineração de dados para incluir outras formas de interação disponíveis no GitHub, como pull requests, revisões de código e eventos de contribuição. Também poderão ser incorporadas novas métricas de análise de redes, algoritmos de caminhos mínimos, estudos temporais da evolução das comunidades e integração com bancos de dados para armazenamento e processamento de maiores volumes de informações.

7.1. Divisão das Atividades

O desenvolvimento do trabalho foi realizado de forma colaborativa entre os integrantes do grupo. Amanda Bicalho Silva foi responsável pela implementação da Etapa 2 e pela elaboração dos slides da apresentação. Karen Joilly Araújo Gregório de Almeida participou da implementação das estruturas de grafos, do desenvolvimento da Etapa 3, das análises realizadas no Gephi e da elaboração do relatório técnico e do diagrama de classes. Pedro Rodrigues Duarte foi responsável pela finalização da Etapa 1 e pela produção do vídeo de demonstração. Tiago Boaventura Amaral desenvolveu a estrutura inicial da Etapa 1, incluindo a implementação da classe principal e a integração dos módulos do sistema.

Referências

- CONSORTIUM, G. (2026). Gephi: The open graph viz platform. Disponível em: <https://gephi.org/>. Acesso em: 27 maio 2026.
- CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., and STEIN, C. (2022). *Introduction to Algorithms*. MIT Press, 4 edition.
- FASTAPI (2026). Typer repository. Disponível em: <https://github.com/fastapi/typer>. Acesso em: 27 maio 2026.